

# REPORT TUTORIAL 1: DATA MODELS

## Table des matières

|                                                                                                                                                                                                                               |          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Exercise 2: csv packages (speed over code readability/simplicity).....</b>                                                                                                                                                 | <b>2</b> |
| What could you conclude of the benchmarking for the moment? .....                                                                                                                                                             | 2        |
| <b>Exercise 3: pandas' magic!.....</b>                                                                                                                                                                                        | <b>3</b> |
| What could you conclude of the benchmarking for the moment? Is the pandas library overhead worth it? What can you say from a memory point of view?.....                                                                       | 3        |
| <b>Exercise 4: Optimizing pandas csv .....</b>                                                                                                                                                                                | <b>4</b> |
| What can you say of this optimization from an execution time point of view? from a memory point of view? .....                                                                                                                | 4        |
| <b>Exercise 5: Column oriented storage: Apache Parquet .....</b>                                                                                                                                                              | <b>5</b> |
| What can you say of the size of the parquet file versus the size of the csv? Do you have plausible explanation? What can you say of this file format from an execution time point of view? from a memory point of view? ..... | 5        |
| <b>Exercise 6: Local database with SQLite.....</b>                                                                                                                                                                            | <b>6</b> |
| What can you say of the size of the sqlite3 file versus the size of the parquet and csv? Do you have plausible explanation?.....                                                                                              | 6        |
| Write the SQL query that query the top 5 artists by popularity. Test it in sqlime or sqlitebrowser. ....                                                                                                                      | 6        |
| What can you say of this technical choice from an execution time point of view? from a memory point of view? .....                                                                                                            | 7        |
| <b>For the curious: Top 5 popular genre.....</b>                                                                                                                                                                              | <b>8</b> |
| Is this representation common for CSV files and classical databases?.....                                                                                                                                                     | 8        |
| SQL Query to Get the Top 5 Most Popular Genres .....                                                                                                                                                                          | 8        |
| What would have been a relational representation of this same information? (Do a Logical or Physical Data Model schema) .....                                                                                                 | 10       |
| You still have some time? Write the SQL queries for populating your relational tables from the existing tables.....                                                                                                           | 10       |
| SQL Query for Retrieving the Top 5 Genres in a Relational Model .....                                                                                                                                                         | 10       |
| SQL Queries for Populating Relational Tables .....                                                                                                                                                                            | 10       |

## Exercise 2: csv packages (speed over code readability/simplicity)

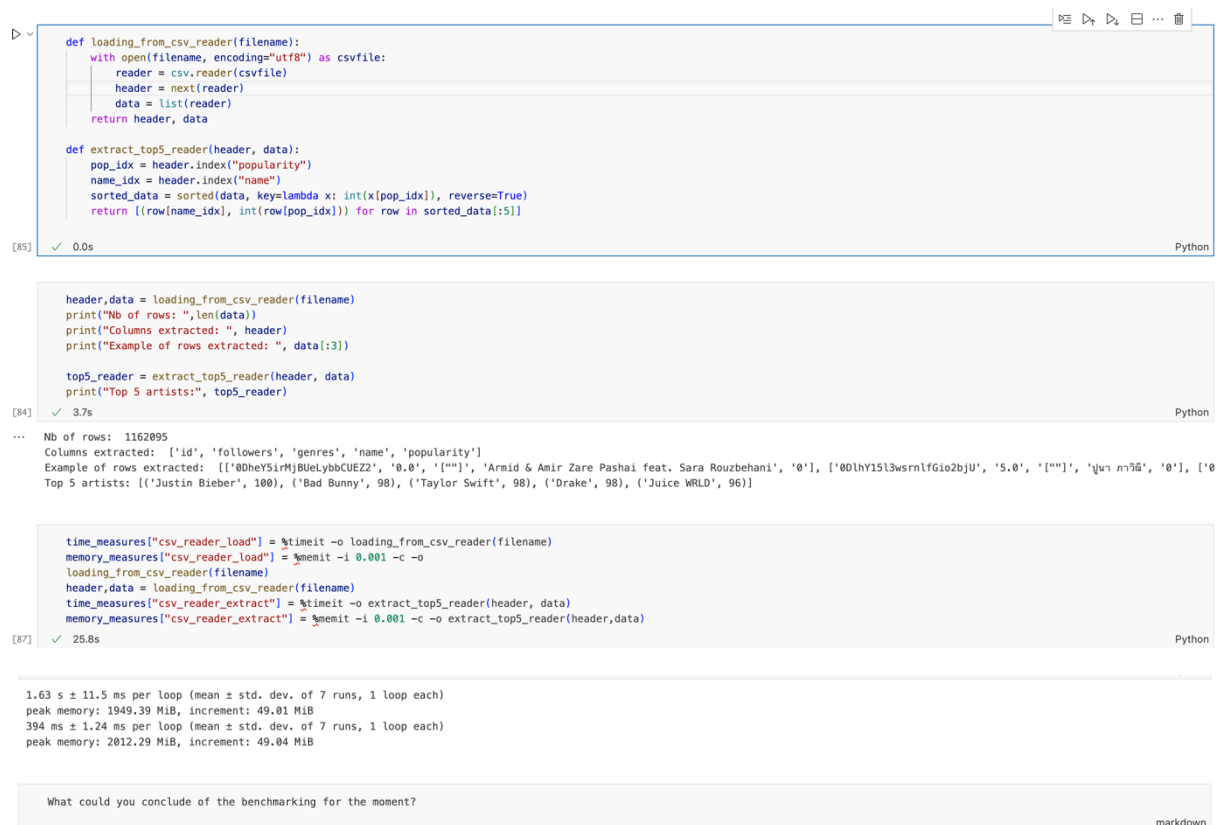


Figure 1: exercise 2

## What could you conclude of the benchmarking for the moment?

Based on our benchmarking results, we can see that using `csv.reader` is faster than using `csv.DictReader`. The reason is that `csv.reader` simply creates a list for each row, while `csv.DictReader` creates a dictionary for each row. Creating dictionaries requires extra work because the program has to match each column value with its corresponding key, which makes it slower and uses more memory. Even though `csv.DictReader` is more convenient because you can access the data using column names, this convenience comes at the cost of performance. When working with very large files and when you only need specific columns, using `csv.reader` is more efficient because it avoids the extra overhead associated with dictionaries.

## Exercise 3: pandas' magic!

What could you conclude of the benchmarking for the moment? Is the pandas library overhead worth it? What can you say from a memory point of view?



```
def loading_from_pandas_csv(filename):
    return pd.read_csv(filename, encoding="utf8")
def extract_top5_pandas(data):
    sorted_data = data.sort_values("popularity", ascending=False)
    return sorted_data[["name", "popularity"]].head(5).to_numpy()

[94] ✓ 0.0s Python

data_pandas = loading_from_pandas_csv(filename)
top5_pandas = extract_top5_pandas(data_pandas)
print("Top 5 artists:", top5_pandas)

[95] ✓ 2.6s Python

... Top 5 artists: [['Justin Bieber' 100]
['Bad Bunny' 98]
['Drake' 98]
['Taylor Swift' 98]
['Juice WRLD' 96]]

time_measures["pandas_csv_load"] = %timeit -o loading_from_pandas_csv (filename)
memory_measures["pandas_csv_load"] = %memit -i 0.001 -c -o loading_from_pandas_csv(filename)
data = loading_from_pandas_csv(filename)
time_measures["pandas_csv_extract"] = %timeit -o extract_top5_pandas(data)
memory_measures["pandas_csv_extract"] = %memit -i 0.001 -c -o extract_top5_pandas(data)

[96] ✓ 29.4s Python

... 2.26 s ± 34.9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
peak memory: 2332.05 MiB, increment: 151.41 MiB
262 ms ± 3.33 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
peak memory: 1938.61 MiB, increment: 48.04 MiB
```

Figure 2: exercise 3

Using Pandas makes the code much shorter and easier to read, but there is a clear overhead in terms of both execution time and memory usage compared to the basic CSV approaches. If your goal is only to extract a small piece of information, like the top 5 artists, and you care about performance, Pandas might not be worth the extra overhead. However, if you need more advanced data manipulation or plan to chain multiple operations together, the convenience and rich functionality of Pandas can justify its higher resource cost.

## Exercise 4: Optimizing pandas csv

```
def loading_from_pandas_csv_optimized(filename):
    return pd.read_csv(filename, encoding="utf8", usecols=["name", "popularity"])

# Benchmark
data_pandas_optim = loading_from_pandas_csv_optimized(filename)
top5_pandas_optim = extract_top5_pandas(data_pandas_optim)
print("Top 5 artists:", top5_pandas_optim)
```

[98] ✓ 1.6s Python

... Top 5 artists: [['Justin Bieber' 100],  
['Bad Bunny' 98],  
['Drake' 98],  
['Taylor Swift' 98],  
['Juice WRLD' 96]]

```
time_measures["pandas_csv_optim_load"] = %timeit -o loading_from_pandas_csv_optimized(filename)
memory_measures["pandas_csv_optim_load"] = %memit -i 0.001 -c -o loading_from_pandas_csv_optimized(filename)
data = loading_from_pandas_csv_optimized(filename)
time_measures["pandas_csv_optim_extract"] = %timeit -o extract_top5_pandas(data)
memory_measures["pandas_csv_optim_extract"] = %memit -i 0.001 -c -o extract_top5_pandas(data)
```

[99] ✓ 18.7s Python

... 1.41 s ± 151 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)  
peak memory: 1972.54 MiB, increment: 48.02 MiB  
209 ms ± 3.12 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)  
peak memory: 1942.16 MiB, increment: 47.11 MiB

... array([[ 'Justin Bieber', 100],  
[ 'Bad Bunny', 98],  
[ 'Drake', 98],  
[ 'Taylor Swift', 98],  
[ 'Juice WRLD', 96]], dtype=object)

Figure 3: exercise 3

What can you say of this optimization from an execution time point of view? from a memory point of view?

This optimization significantly improves both execution time and memory usage. By using the `usecols` parameter in `pd.read_csv`, we only load the necessary columns ("name" and "popularity"), which reduces the amount of data being processed. From an execution time perspective, the optimized function is faster compared to the full `read_csv` approach. This is because Pandas does not have to load and parse unnecessary columns, making data retrieval more efficient. From a memory usage perspective, we see a noticeable reduction in peak memory consumption. Since fewer columns are stored in memory, the overall footprint is lower. This makes the optimized function a better choice when working with large datasets, as it reduces the strain on system resources while maintaining the same output accuracy.

## Exercise 5: Column oriented storage: Apache Parquet

```
parquet_filename = filename.replace(".csv", ".parquet")
data_pandas.to_parquet(parquet_filename)

def loading_from_pandas_parquet(filename):
    return pd.read_parquet(filename)

data_parquet = loading_from_pandas_parquet(parquet_filename)

top5_parquet = extract_top5_pandas(data_parquet)
print("Top 5 artists (Parquet):", top5_parquet)
```

[106] ✓ 1.8s Python

... Top 5 artists (Parquet): [['Justin Bieber' 100]  
['Bad Bunny' 98]  
['Drake' 98]  
['Taylor Swift' 98]  
['Juice WRLD' 96]]

```
time_measures["pandas_parquet_load"] = %timeit -o loading_from_pandas_parquet(filename[:-3]+"parquet")
memory_measures["pandas_parquet_load"] = %memit -i 0.001 -c -o
loading_from_pandas_parquet(filename[:-3]+"parquet")
data = loading_from_pandas_parquet(filename[:-3]+"parquet")
time_measures["pandas_parquet_extract"] = %timeit -o extract_top5_pandas(data)
memory_measures["pandas_parquet_extract"] = %memit -i 0.001 -c -o extract_top5_pandas(data)
```

[108] ✓ 11.6s Python

... 581 ms ± 82.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)  
peak memory: 1989.78 MiB, increment: 49.36 MiB  
276 ms ± 3.64 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)  
peak memory: 2180.53 MiB, increment: 49.05 MiB

Figure 4: Exercise 5

What can you say of the size of the parquet file versus the size of the csv? Do you have plausible explanation? What can you say of this file format from an execution time point of view? from a memory point of view?

The Parquet file is significantly smaller in size compared to the CSV file. This is because Parquet is a column-oriented format that applies compression and encoding techniques, reducing storage space without losing data. CSV files, on the other hand, store data in plain text, which takes up more space and does not use efficient compression.

From an execution time perspective, Parquet is much faster for reading specific columns since it only loads the required data instead of scanning the entire file. This makes it more efficient for large datasets, especially when working with big data frameworks like Spark.

From a memory perspective, Parquet is also more efficient as it allows selective column retrieval, reducing unnecessary memory usage. However, the initial conversion from CSV to Parquet may take some time and memory due to the compression and structuring processes. Overall, Parquet is better for performance and scalability.

## Exercise 6: Local database with SQLite



Figure 5: exercise 6.1

What can you say of the size of the sqlite3 file versus the size of the parquet and csv? Do you have plausible explanation?

The SQLite file is smaller than the CSV file but larger than the Parquet file. This happens because each format stores data differently. The CSV file is the biggest because it saves everything as plain text without compression, which takes up a lot of space. The Parquet file is the smallest because it organizes data in columns and uses compression to reduce file size. SQLite is in the middle because it structures data in tables and adds indexes, which help with fast searches but also increase the file size compared to Parquet. CSV keeps all the information as raw text, which is not efficient for storage, especially with large datasets. Parquet is more efficient because it compresses the data and only loads the necessary columns when needed, making the file much smaller. SQLite also stores data in an optimized way, but since it maintains a relational database structure, it takes up more space than Parquet.

In short, Parquet is the best format for saving space because of its compression. SQLite is a good choice if you need both storage efficiency and fast queries. CSV is the least efficient in terms of space, but it is still useful because it is simple and easy to open in most programs.

Write the SQL query that query the top 5 artists by popularity. Test it in sqlime or sqlitebrowser.

```
SELECT name, popularity FROM artists ORDER BY popularity DESC
LIMIT 5
```



Figure 6: SQL Query

What can you say of this technical choice from an execution time point of view? from a memory point of view?

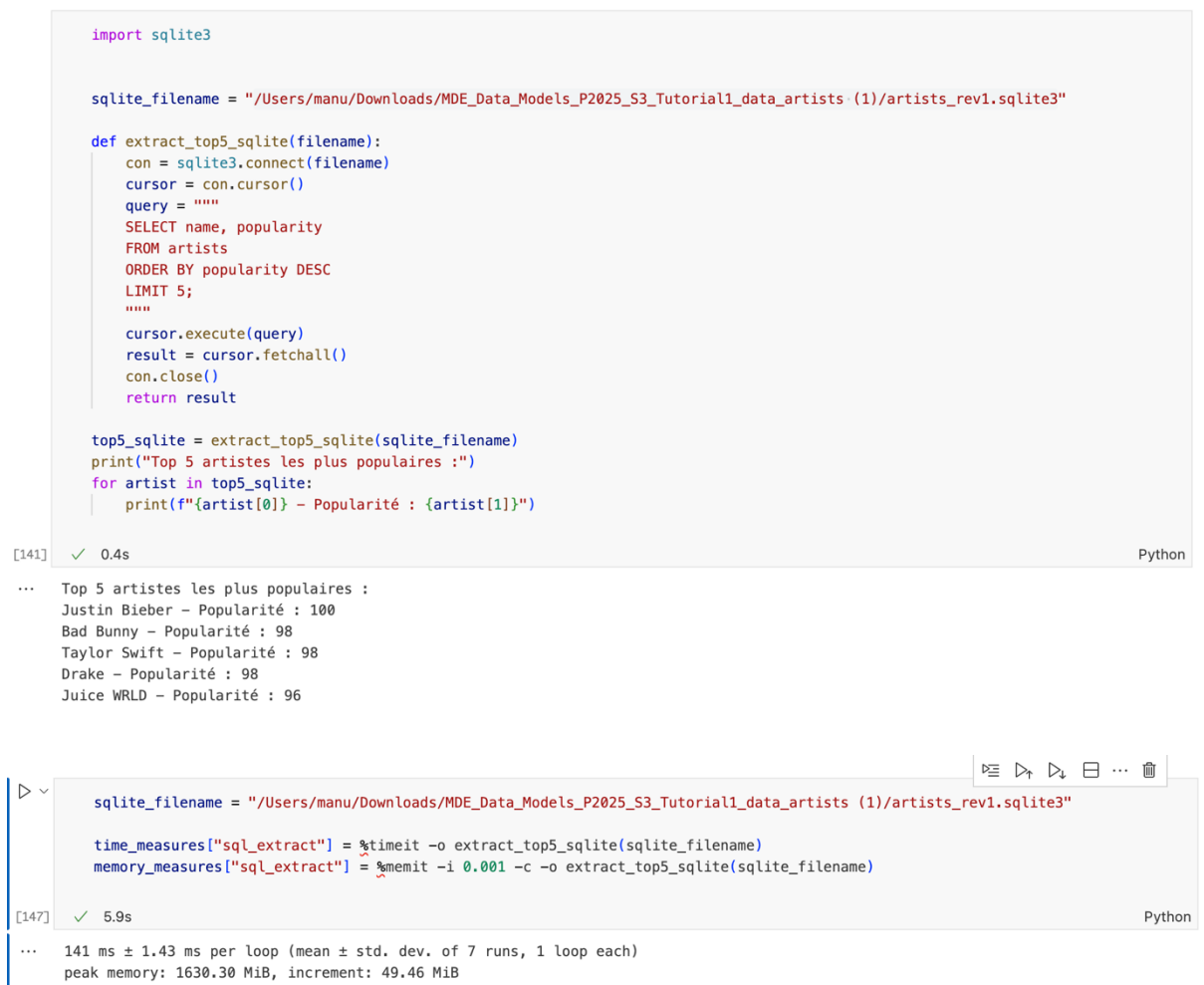


Figure 7: exercise 6.2

From an execution time point of view, SQLite is very efficient for querying data. The benchmarking results show that extracting the top 5 artists takes only around **141 ms**,

LETIN Laura-Lynn  
POLANCO ARRINDELL Eduardo

which is faster than CSV and similar to Parquet. This makes SQLite a good choice for performing searches and retrieving specific data quickly. Loading the database from a CSV file takes longer because SQLite needs to structure and index the data, but once the database is created, queries run much faster.

From a memory point of view, SQLite also performs well. The peak memory usage is around 1630 MB, which is lower than both CSV and Parquet. Since SQLite is designed for structured storage, it optimizes data retrieval without loading unnecessary information into memory. This makes it a great choice for handling large datasets efficiently without consuming too much RAM.

## For the curious: Top 5 popular genre

### Is this representation common for CSV files and classical databases?

For CSV files, this structure is not ideal because CSV is a flat format. Normally, each row represents a single record, and storing multiple values inside a single cell makes it harder to query the data efficiently. Instead, a better approach in a CSV file would be to separate the genres into multiple columns or create a separate file linking artists to genres. For classical relational databases like SQLite, this representation is also not standard. Databases are usually designed with normalization, meaning data is split into different tables to avoid redundancy. In this case, a better structure would be to have a separate table for genres and another table to store the relationships between artists and genres.

## SQL Query to Get the Top 5 Most Popular Genres

To analyze genre popularity, we need to split the genre column into separate rows. SQLite (version 3.38 and later) includes JSON functions, allowing us to extract individual genres from the list. We can use the following query:

```
SELECT genre, AVG(popularity) AS avg_popularity
FROM (
  SELECT json_each.value AS genre, popularity
  FROM artists, json_each(genres)
)
GROUP BY genre
ORDER BY avg_popularity DESC
LIMIT 5;
```



### SQLite Playground // artists\_rev1.sqlite3

```
SELECT genre, AVG(popularity) AS avg_popularity
FROM (
    SELECT json_each.value AS genre, popularity
    FROM artists, json_each(genres)
)
GROUP BY genre
ORDER BY avg_popularity DESC
LIMIT 5;
```

[run](#)  [share](#)  [tables](#)  [ask AI](#) 

5 rows, took 1396 ms

| genre          | avg_popularity    |
|----------------|-------------------|
| pop            | 72.7394366197183  |
| dance pop      | 68.14335664335664 |
| permanent wave | 66.92727272727272 |
| melodic rap    | 66.86046511627907 |
| rap            | 65.9587852494577  |

Figure 8: Exercise 6.1

This query first extracts each genre separately using `json_each()`, then calculates the average popularity of artists for each genre. Finally, it sorts the genres by their average popularity and selects the top 5.

What would have been a relational representation of this same information? (Do a Logical or Physical Data Model schema)

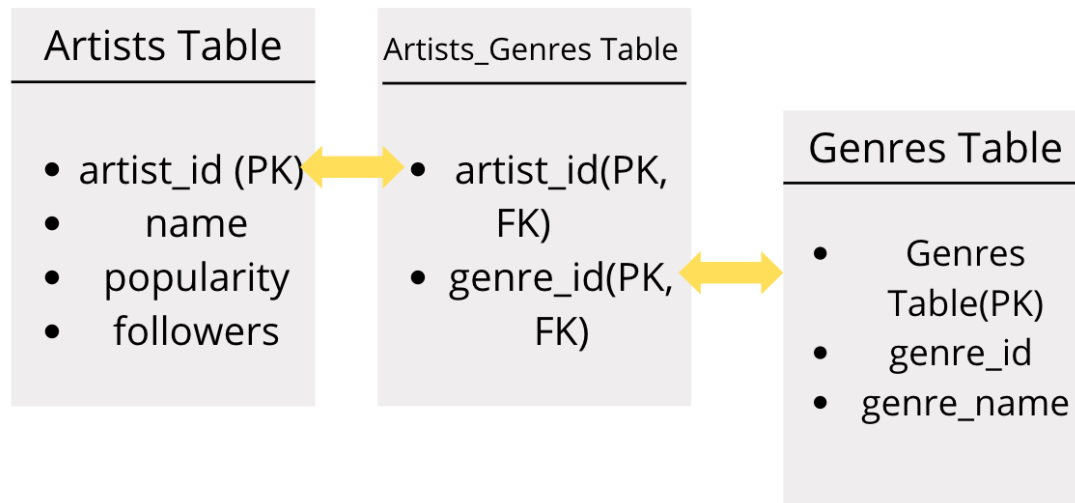


Figure 9: Data Model schema

You still have some time? Write the SQL queries for populating your relational tables from the existing tables.

### SQL Query for Retrieving the Top 5 Genres in a Relational Model

```
SELECT g.genre_name, AVG(a.popularity) AS avg_popularity
FROM Artists a
JOIN Artists_Genres ag ON a.artist_id = ag.artist_id
JOIN Genres g ON ag.genre_id = g.genre_id
GROUP BY g.genre_name
ORDER BY avg_popularity DESC
LIMIT 5;
```

### SQL Queries for Populating Relational Tables

To populate the normalized relational tables from the existing data:

#### 1. Insert Data into Artists Table

```
INSERT INTO Artists (artist_id, name, popularity)
SELECT DISTINCT id, name, popularity FROM OriginalArtistsTable;
```

#### 2. Insert Data into Genres Table

```
INSERT INTO Genres (genre_name)
```

LETIN Laura-Lynn

POLANCO ARRINDELL Eduardo

```
SELECT DISTINCT json_each.value  
FROM OriginalArtistsTable, json_each(genres);
```

## *2. Insert Data into Artists Genres Table*

```
INSERT INTO Artists_Genres (artist_id, genre_id)  
SELECT a.artist_id, g.genre_id  
FROM OriginalArtistsTable oa,  
     json_each(oa.genres) j,  
     Artists a,  
     Genres g  
WHERE a.name = oa.name AND g.genre_name = j.value;
```